Final Report: The Automaton Auditor

Week 2 — TRP1 Challenge: Orchestrating Deep LangGraph Swarms for Autonomous Governance

---

1. Executive Summary

This report documents the **final implementation** of the Automaton Auditor: a hierarchical LangGraph swarm that audits Week 2 repositories and their PDF reports. The system implements the full “Digital Courtroom”: **Layer 1** (Detectives: RepoInvestigator, DocAnalyst, VisionInspector) collect forensic evidence in parallel; **Layer 2** (Judicial Panel: Prosecutor, Defense, Tech Lead) produce structured opinions per rubric criterion; **Layer 3** (Chief Justice) applies hardcoded conflict-resolution rules and outputs a Markdown audit report.

Deliverables are in place: typed state with reducers, sandboxed repo tools with AST-based analysis, PDF ingestion and image extraction, three judge personas with `.with_structured_output(JudicialOpinion)`, Chief Justice synthesis with Rule of Security, Fact Supremacy, and Functionality weight, and end-to-end graph from repo URL + PDF to written report. Remaining gaps (e.g. conditional edges for “Evidence Missing”, optional vision-model analysis of diagrams) are documented with a remediation plan.

Overall self-audit aggregate score: The peer's agent (see `audit/report_bypeer_received/audit_report_ahmed.md`) audited this repository and produced an **overall score of 2.70/5**, with a security-related cap at 3. This serves as our primary external assessment.

Most impactful findings from the peer feedback loop: (1) **Safe Tool Engineering (2/5)** — The peer's Prosecutor and Tech Lead reported insufficient evidence of sandboxing and flagged possible use of `os.system`; this triggered the security cap and is the highest-impact finding. (2) **Structured Output Enforcement (1/5)** — The peer's agent did not find `src/nodes/judges.py` in the cloned repo (path or clone-scope issue), leading to the lowest criterion score. (3) **Judicial Nuance (2/5)** and **Chief Justice Synthesis** — The peer noted insufficient evidence of distinct judge personas and deterministic synthesis rules. These outcomes directly informed our remediation plan (prioritized below) and our reflection on how to improve both this repo and our auditor's detection of the same issues in others.

---

2. Architecture Deep Dive

2.1 Dialectical Synthesis (Thesis–Antithesis–Synthesis)

What it is: The rubric requires a *dialectical* process: not a single "grader" but three distinct viewpoints that argue over the same evidence, with a final synthesis that resolves conflict.

How we implement it:

- **Thesis / Antithesis:** Three judge personas in `src/nodes/judges.py` each receive the *same* evidence summary (from `_evidence_summary_for_judges`) and the same rubric dimensions. They differ only by system prompt:
    - **Prosecutor:** "Trust No One. Assume Vibe Coding." — scrutinizes gaps, security, laziness; argues for low scores when requirements are unmet.
    - **Defense:** "Reward Effort and Intent. Look for the Spirit of the Law." — highlights effort, Git history, and architectural intent even when implementation is imperfect.
    - **Tech Lead:** "Does it actually work? Is it maintainable?" — tie-breaker; focuses on artifacts, reducers, and safety; gives 1, 3, or 5 with remediation advice.

- **Synthesis:** `src/nodes/justice.py` implements **Chief Justice** as *deterministic Python logic*, not an LLM. It groups `state.opinions` by `criterion_id` and, for each criterion, applies:
    - **Rule of Security:** If the Prosecutor's argument mentions security-related terms (e.g. `os.system`, injection), the score is capped at 3 regardless of Defense.
    - **Rule of Functionality:** For architecture-related criteria (`graph_orchestration`, `state_management_rigor`, `safe_tool_engineering`), the Tech Lead's score is taken as the final score.
    - **Variance > 2:** If the three scores differ by more than 2, a dissent summary is written and the Tech Lead is used as tie-breaker (or median).

So "Dialectical Synthesis" is implemented by: (1) three independent opinions on the same evidence, (2) a fixed rule set that resolves conflict (security overrides effort, Tech Lead breaks ties and dominates on architecture), and (3) explicit dissent in the report when variance is high.

2.2 Fan-In / Fan-Out and State Synchronization

Fan-out: Multiple nodes are triggered from one point and run in parallel so that work is split without sequential bottlenecks.

How we implement it:

- **Detective fan-out:** In `src/graph.py`, `add_conditional_edges(START, lambda _: ["repo_investigator", "doc_analyst", "vision_inspector"])` causes all three detective nodes to be invoked from START. LangGraph runs these branches in parallel; each returns updates to `evidences` keyed by source (`repo_investigator`, `doc_analyst`, `vision_inspector`).

- **State merge (no overwrite):** `AgentState` in `src/state.py` declares `evidences` as `Annotated[Dict[str, List[Evidence]], operator.ior]`. The `operator.ior` reducer merges dicts (e.g. `dict_a | dict_b`), so each detective's output is merged by key instead of overwriting. Similarly, `opinions` is `Annotated[List[JudicialOpinion], operator.add]`, so multiple judges (if we split them into separate graph nodes later) would append rather than replace.

Fan-in: All parallel branches must complete before the next stage uses their combined result.

- **Evidence aggregation:** The graph has a single **EvidenceAggregator** node. All three detectives have edges *into* it (`repo_investigator → evidence_aggregator`, `doc_analyst → evidence_aggregator`, `vision_inspector → evidence_aggregator`). LangGraph's semantics ensure the aggregator runs only after all incoming edges have been updated; at that point `state.evidences` contains merged evidence from all three detectives.

- **Judicial panel:** The three judges are currently invoked *inside* one node (`judicial_panel_node`) in sequence (to keep one LLM call per persona and avoid rate limits). They all read the same aggregated evidence and append to `state.opinions` via the `operator.add` reducer. So conceptually we have "three views" fanning into one list, then Chief Justice runs once on that list.

State synchronization is thus: (1) fan-out from START to three detectives, (2) reducer `operator.ior` merges their `evidences`, (3) one aggregator node as sync point, (4) judicial panel produces `opinions` (reducer `operator.add`), (5) Chief Justice reads the full `opinions` and writes `final_report` once.

2.3 Metacognition

What it is (in this context): The system is not just "generating" code or text; it is *evaluating* artifacts (repo + report) against a rubric. That requires "thinking about thinking": understanding what "good" means (success/failure patterns), distinguishing fact from opinion, and applying rules consistently.

How we implement it:

- **Constitution (rubric):** `config/rubric.json` holds the grading constitution: dimensions with `forensic_instruction`, `success_pattern`, and `failure_pattern`. Detectives are given only the dimensions whose `target_artifact` matches their role (repo vs PDF vs images). Judges receive the same dimensions plus evidence; they are prompted to score against these patterns. The Chief Justice does not re-interpret the rubric; it applies fixed rules (security, functionality, variance) to the numeric and textual output of the judges.

- **Fact vs. opinion:** Detectives output only **Evidence** (goal, found, location, rationale, confidence, content). They do not score. Scoring is done by the Judicial Panel, which *cites* evidence in `cited_evidence` and gives an argument. The Chief Justice then uses facts (e.g. "Prosecutor mentioned security") to override or weight opinions. So the pipeline enforces: facts first (detectives), then interpretation (judges), then rule-based resolution (Chief Justice).

- **Self-evaluation:** Running the auditor against its own repo (and this report) is an instance of metacognition: the system's design is documented and then assessed by the same rubric it uses to assess others. The final report (e.g. in `audit/report_onself_generated/`) is the tangible output of that self-audit.

---

3. Architectural Diagram: StateGraph Flow

The following diagram reflects the actual wiring in `src/graph.py`.

INPUT: AgentState { repo_url, pdf_path, rubric_dimensions, evidences={}, opinions=[], final_report=None }

```
┌──────────────────────────┐
│      START       │
└──────────┬───────────────┘
           │
┌──────────┴──────────┬──────────┴──────────┐
│                     │                     │
▼                     ▼                     ▼
```

```
┌──────────────────────┐  ┌──────────────────────┐
┌──────────────────────┐

│ RepoInvestigator│  │  DocAnalyst   │  │ VisionInspector │  ← Fan-out (parallel)
│ (clone, git log,│  │ (ingest PDF,  │  │ (extract images │
│  AST graph/state│  │ RAG query,    │  │  from PDF;      │
│  sandbox check) │  │ path extract) │  │  optional vision)│
└──────────┬───────────┘  └──────────┬───────────┘  └──────────┬──────────┘
           │              │              │
           │ evidences    │ evidences    │ evidences
           │ (operator.ior merge) │              │
           └──────────────┴──────────────┘
                        ▼
           ┌──────────────────────────┐
           │  EvidenceAggregator  │  ← Fan-in (sync)
           │  (pass-through; state  │
           │   now has full       │
           │   evidences)         │
           └──────────┬───────────────┘
                      │
                      ▼
           ┌──────────────────────────┐
           │  Judicial Panel      │
           │  (Prosecutor, Defense, │
```

```
│   Tech Lead; each      │
│   returns opinions     │
│   via structured output)│
└──────────────┬──────────────┘
               │ opinions (operator.add)
               ▼
┌──────────────────────────────┐
│  Chief Justice        │
│  (hardcoded rules:      │
│   security, functionality,│
│   variance → dissent)   │
└──────────────┬──────────────┘
               │ final_report: AuditReport
               ▼
              END
               │
               ▼
Output: Markdown file (audit/report_*.md)
```

Data flow summary:

- **Edges** carry the full `AgentState`; only the keys that a node updates are specified (e.g. `evidences`, `opinions`, `final_report`).
- **Reducers** ensure parallel nodes do not overwrite: `evidences` uses `operator.ior`, `opinions` uses `operator.add`.

- **Single writer:** Only the Chief Justice writes `final_report`; no reducer needed there.

---

4. Criterion-by-Criterion Self-Audit (Implementation vs. Rubric)

4.1 Implementation and self-assessment

Criterion	Implementation	Self-assessment
Git Forensic Analysis	`repo_tools.extract_git_history()` runs `git log --oneline --reverse --format=%h %s %ci`; RepoInvestigator checks commit count and progression keywords.	Meets: git log extracted; progression story checked.
State Management Rigor	`state.py`: `Evidence`, `JudicialOpinion`, `CriterionResult`, `AuditReport` (Pydantic); `AgentState` (TypedDict) with `Annotated[…, operator.ior]` and `operator.add`. `repo_tools.analyze_state_management()` uses AST to find BaseModel/TypedDict, Evidence, JudicialOpinion, reducers.	Meets: typed state and reducers; AST verification.
Graph Orchestration	`graph.py`: StateGraph with conditional_edges from START to three detectives; edges into EvidenceAggregator; then judicial_panel → chief_justice → END. `analyze_graph_structur`	Meets: detective fan-out/fan-in; judicial panel and Chief Justice in graph.

Criterion	Implementation	Self-assessment
	`e()` parses add_edge/add_conditional_edges and sync node names.	
Safe Tool Engineering	`clone_repo()` requires a target_dir (used with `tempfile.TemporaryDirectory()` in the node); `subprocess.run` for git; URL validated with HTTPS GitHub regex; no `os.system`. `check_sandboxed_tools()` scans for tempfile/subprocess and flags os.system.	Meets: sandboxed clone, subprocess, no os.system.
Structured Output Enforcement	Judges use `ChatOpenAI(...).with_structured_output(JudgeVerdict)`; JudgeVerdict contains list of opinions with criterion_id, score, argument, cited_evidence; converted to `JudicialOpinion` with judge name. Retry/fallback on failure returns default opinions.	Meets: Pydantic-bound LLM output; schema alignment.
Judicial Nuance	Prosecutor, Defense, Tech Lead have distinct system prompts (adversarial vs. forgiving vs. pragmatic). Same evidence and dimensions; different scores/arguments expected.	Meets: three personas; dialectical design.
Chief Justice Synthesis	`justice.py`: `_resolve_score()` implements Rule of Security	Meets: deterministic rules; Markdown report.

Criterion	Implementation	Self-assessment
	(cap 3), Rule of Functionality (Tech Lead for arch criteria), variance > 2 → dissent and tie-break. Output is `AuditReport`; `audit_report_to_markdown()` produces Markdown.	
Theoretical Depth (Doc)	DocAnalyst uses `query_pdf_chunks()` for terms like “Dialectical Synthesis”, “Metacognition”; evidence includes chunk excerpts.	Meets: RAG-lite over PDF; concept search.
Report Accuracy	`extract_file_paths_from_text()` pulls paths from PDF; evidence lists them for cross-check with RepoInvestigator. No automatic “verified vs hallucinated” list in report yet.	Partial: path extraction and evidence; cross-ref could be explicit in report.
Swarm Visual (Diagram)	VisionInspector: `extract_images_from_pdf()` in doc_tools; node returns Evidence for diagram dimension. Optional vision model not wired; implementation present, execution optional per rubric.	Meets: image extraction implemented; vision analysis optional.

4.2 Dialectical tension: judge opinions and conflicts (from peer audit of this repository)

The following shows how each of the three judge personas (Prosecutor, Defense, Tech Lead) assessed this repository when audited by the peer's agent (`audit/report_bypeer_received/audit_report_ahmed.md`). Disagreements and dissent are explicit.

Criterion	Prosecutor	Defense	Tech Lead	Final	Conflict / dissent
Git Forensic Analysis	3/5 — "Lack of evidence regarding meaningful commit messages."	4/5 — "Six distinct commits, clear progression; atomic sequence aligns with success pattern."	5/5 — "Structured commit history; clear progression."	3/5	Defense and Tech Lead reward progression; Prosecutor withholds for message quality.
State Management Rigor	4/5 — "TypedDict AgentState with Pydantic; reducers present."	4/5 — "Pydantic and TypedDict; thoughtful engineering."	5/5 — "Best practices; Pydantic, TypedDicts, reducers."	3/5	All positive; Tech Lead strongest.
Graph Orchestration	2/5 — "Insufficient evidence for synchronization nodes."	4/5 — "StateGraph with multiple edges; more sync evidence would help."	3/5 — "Basic fan-out/fan-in; limited edges; improve error handling."	3/5	Prosecutor 2 vs Defense 4; Tech Lead tie-break (3). Dissent on sync visibility.
Safe Tool Engineering	1/5 — "No sandboxing; raw os.system; security concerns."	2/5 — "Lacked sandboxing; os.system concerning."	2/5 — "No temp dir; raw os.system; recommend subprocess."	2/5	All critical; security cap. Highest-impact finding.
Structured Output Enforcement	1/5 — "judges.py missing in cloned repo."	2/5 — "Absence of judges.py; intent exists logically."	1/5 — "No evidence; judges file missing."	1/5	Agreement on missing evidence. Lowest score.
Judicial Nuance	1/5 — "No evidence;	3/5 — "Distinct	2/5 — "Insufficient	2/5	Prosecutor/Tech Lead

Criterion	Prosecutor	Defense	Tech Lead	Final	Conflict / dissent
	lacks persona separation."	judges; prompt design speculative."	evidence for distinct personas."		demand evidence; Defense partial credit.
Chief Justice Synthesis	1/5 — "No detected deterministic rules."	3/5 — "Deterministic logic implied; further proof would help."	3/5 — "Partial deterministic rules; stricter rules would help."	3/5	Prosecutor sees no rules; Defense/Tech Lead partial.
Theoretical Depth	2/5 — "Terms insufficiently integrated; superficial."	4/5 — "Significant terms in meaningful contexts."	4/5 — "Terminology with context; deeper integration would help."	4/5	Prosecutor flags keyword-dropping; Defense/Tech Lead reward substance.
Report Accuracy	2/5 — "Absence of verifying path existence; hallucination risk."	4/5 — "Claims generally align with repo."	4/5 — "Cross-reference shows alignment."	4/5	Prosecutor demands verification; others accept alignment.
Swarm Visual	1/5 — "Inadequate evidence for parallelism visualization."	2/5 — "Absence of visual diagrams hampers."	2/5 — "Absence of diagrams reduces clarity."	2/5	Agreement that diagram evidence is missing.

---

5. Reflection on the MinMax Feedback Loop

5.1 Findings from our audit of the peer's repository

When our agent audited our assigned peer's repository (Ahmed-Hibet/automaton-auditor), the **detective layer executed successfully**: the repo was cloned, `git log` and AST-based checks (graph structure, state management, sandboxing) ran, and PDF ingestion and path

extraction ran where a report was provided. The **judicial panel** could not complete due to API quota limits (429) during that run, so the reported scores were fallback defaults (3/5 per criterion). In a successful run, our agent would have produced: (1) criterion-by-criterion scores with distinct Prosecutor, Defense, and Tech Lead opinions and cited evidence; (2) dissent summaries where score variance exceeded 2; (3) remediation per dimension tied to rubric success/failure patterns; (4) Safe Tool Engineering and Structured Output evidence (e.g. presence of `src/nodes/judges.py`, use of `tempfile`/`subprocess` vs `os.system`). The peer's repo was cloneable and our pipeline produced a structured Markdown report, demonstrating that our auditor runs end-to-end on an external target. We would re-run the audit with sufficient API quota to surface real dialectical tension and peer-specific findings to complete the MinMax loop.

5.2 What a peer's agent caught in our work

- **What a peer's agent might catch:** (1) Missing conditional edges for "Evidence Missing" or "Node Failure" (graph always proceeds to judges). (2) Judicial panel runs three LLM calls in sequence, not as three separate graph nodes, so the "Judges fan-out" is conceptual rather than explicit in the graph. (3) Report Accuracy dimension could output an explicit "Verified vs. Hallucinated paths" section. (4) Vision model (e.g. GPT-4o/Gemini) not invoked for diagram classification. (5) **As observed in peer audit:** Safe Tool Engineering (2/5) and security cap; Structured Output (1/5) due to judges.py not found; Judicial Nuance (2/5) and Chief Justice evidence gaps.

5.3 How we could update the agent after peer feedback

- **How we could update the agent after peer feedback:** (1) Add conditional edge from EvidenceAggregator: if critical evidence is missing (e.g. repo clone failed), route to a "retry" or "fail_gracefully" node instead of judicial panel. (2) Optionally split judicial_panel into three graph nodes (Prosecutor, Defense, TechLead) with edges into a "judge_aggregator" so the rubric's "Judges in parallel" is visible in the graph. (3) In Chief Justice or DocAnalyst, add a step that compares paths mentioned in the PDF to RepoInvestigator's file list and writes "Verified paths" / "Hallucinated paths" into the report. (4) In VisionInspector, call a vision API on extracted images when available and add diagram-classification evidence. (5) Ensure our repo structure and clone depth make `src/nodes/judges.py` discoverable by peer agents.

---

6. Remediation Plan for Remaining Gaps

Remediation items are **prioritized by impact and dependency**: P1 (blocking/security and highest peer-impact) first, then P2 (core rubric visibility), then P3 (quality and report clarity). Each item is **linked to the rubric dimension** it addresses.

Priority	Rubric dimension(s)	Gap	Remediation
P1	**Safe Tool Engineering**	Peer audit reported insufficient sandboxing evidence and possible `os.system`; security cap applied.	Ensure `src/tools/repo_tools.py` is scanned correctly; document sandboxing in README; strengthen AST evidence so forensic tools detect tempfile/subprocess reliably.
P1	**Structured Output Enforcement**	Peer's agent did not find `src/nodes/judges.py` (score 1/5).	Verify repo layout and clone depth so `src/nodes/judges.py` is present; ensure RepoInvestigator scans `src/nodes/`; add explicit evidence for `.with_structured_output(JudicialOpinion)`.
P2	**Graph Orchestration**	No conditional edges for clone/PDF failure.	Add router after EvidenceAggregator: if evidences lack repo_investigator success, route to partial_report or error_report node; still produce minimal AuditReport.
P2	**Judicial Nuance**, **Chief Justice Synthesis**	Judges as one node; peer noted insufficient evidence of personas and deterministic rules.	Refactor to three graph nodes (prosecutor, defense, tech_lead) with judge_sync; document Chief Justice rules in

Priority	Rubric dimension(s)	Gap	Remediation
			code/config for forensic detection.
P3	**Report Accuracy**	No explicit verified/hallucinated path list.	Intersect paths from PDF with RepoInvestigator file list; write Verified paths and Hallucinated paths into report.
P3	**Swarm Visual**	VisionInspector does not call vision model.	Add optional vision API call when images exist and API key set; append diagram-classification to Evidence.
P3	Report quality	Remediation section repetitive.	In `audit_report_to_markdown()`, deduplicate or summarize remediation bullets per criterion.

---

7. Deliverables Checklist

- **Source:** `src/state.py`, `src/tools/repo_tools.py`, `src/tools/doc_tools.py`, `src/nodes/detectives.py`, `src/nodes/judges.py`, `src/nodes/justice.py`, `src/graph.py`, `main.py`, `config/rubric.json`, `pyproject.toml`, `.env.example`, `README.md`, optional `Dockerfile`.
- **Audit reports:** `audit/report_onself_generated/`, `audit/report_onpeer_generated/`, `audit/report_bypeer_received/` (run with `--output` to populate).